

Parallel Decision Diagram-based Multiobjective Discrete Optimization

Rahul Patel¹, Elias B. Khalil¹

¹Department of Mechanical and Industrial Engineering, University of Toronto, Canada
rm.patel@mail.utoronto.ca, elias.khalil@utoronto.ca

Abstract

Decision diagrams (DDs) have recently emerged as a state-of-the-art technique for solving multiobjective discrete optimization problems by quickly enumerating their Pareto frontier (PF). Existing implementations rely on serial algorithms for PF enumeration, where the primary computational bottleneck arises from repeatedly performing dominance filtering at each node of the DD. This work studies the impact of parallelizing dominance filtering on PF enumeration time. We evaluate the parallel algorithms on benchmark instances of the multiobjective knapsack, set packing, and traveling salesperson problems. Across instance sizes, the maximum speed-up achieved by a GPU variant is $7.49\times$ for MOKP, $15.08\times$ for MOSPP, and $13.60\times$ for MOTSP relative to the state-of-the-art serial implementation. The maximum speed-up achieved by the best parallel CPU variant is $3.36\times$ for MOKP, $2.68\times$ for MOSPP, and $5.75\times$ for MOTSP relative to the same baseline, with the caveat that it is able to solve more instances than the GPU variant. The code is available at <https://anonymous.4open.science/r/PDD>.

1 Introduction

Multiobjective discrete optimization (MODO) (???) problems arise naturally in many applications where conflicting criteria need to be considered simultaneously in conjunction with constraints and integer variables. While problem-specific (?) and generic evolutionary heuristics (?) have been developed for this problem class, recovering the exact Pareto frontier (PF) is desirable in critical applications such as healthcare, finance, and infrastructure planning. Over the past decade, decision diagrams (DDs) (?) have gained traction as an effective framework for solving discrete problems that can be cast in the language of dynamic programming. As a data structure, the DD is an implicit representation of all feasible solutions to a problem. Naturally, one can enumerate the subset of such feasible solutions that are Pareto-optimal by using the multicriteria-shortest path algorithm on the network represented by a DD (?). This approach has been shown to be quite efficient in practice, yielding exact Pareto frontiers rather quickly for problems with 2-7 objectives and tens to hundreds of decision variables (?).

Despite the success of DD-based solvers, extracting the exact Pareto frontier remains computationally demanding. The primary bottleneck lies in the state-space expansion, specifically the localized Pareto dominance filtering performed at each node to discard suboptimal labels. Because dominance checking requires pairwise comparisons among all candidate objective vectors at a node, its worst-case time complexity grows quadratically with the size of the intermediate frontiers. As the number of objectives or the size of the problem increases, this filtering step becomes the bottleneck, limiting the applicability of serial DD solvers.

Simultaneously, the proliferation of graphics processing units (GPUs) due to the rise of deep learning has prompted optimization researchers to reconsider classical algorithms and attempt to parallelize them for improved scalability (?). GPUs are throughput-oriented processors designed to execute thousands of concurrent threads. The massive amount of independent, pairwise vector comparisons required during dominance filtering makes this specific bottleneck highly amenable to the *Single Instruction Multiple Thread* (SIMT) architecture of GPUs.

To address this computational bottleneck, we introduce the first parallel DD-based framework for MODO. Through a careful analysis of the PF enumeration process, we identify opportunities for both node-level and candidate-level parallel computation. We propose static and dynamic resource allocation schemes that take into account the nature of the hardware being used (central processing unit (CPU) or GPU) to efficiently balance the varying workloads across the DD layers.

The main contributions of this paper are summarized as follows:

- We present the first parallel algorithm for exact PF enumeration on DDs, identifying key opportunities for concurrent state expansion and dominance filtering.
- We design and implement novel static and dynamic memory and thread allocation schemes tailored for both multi-core CPUs and massively parallel GPUs, effectively mitigating the hazards of data races and synchronization overhead.
- We conduct extensive computational experiments on benchmark instances of the multiobjective knapsack, set packing, and traveling salesperson problems with up to

seven objectives.

- We demonstrate that our parallelization schemes achieve substantial reductions in wall-clock time compared to the state-of-the-art serial enumeration baseline, achieving speedups of up to $\times$ on CPUs and $\times$ on GPUs.

The remainder of this paper is organized as follows. Section 2 provides the necessary background on MODO, DDs, and GPU architecture. Section 3 details our parallel enumeration methodology and resource allocation schemes. Section 4 presents the computational evaluation across various problem classes. The related work is provided in Section 5. Finally, Section 6 concludes the paper and discusses avenues for future work.

2 Preliminaries

2.1 Multiobjective Discrete Optimization

A MODO problem can be defined as the minimization of an m -dimensional objective vector function $\mathbf{f}(\mathbf{x}) = (f_1(\mathbf{x}), f_2(\mathbf{x}), \dots, f_m(\mathbf{x}))^\top$ subject to $\mathbf{x} \in \mathcal{X}$. Here, $\mathbf{x} = (x_1, x_2, \dots, x_n)^\top$ represents a vector of discrete decision variables, and $\mathcal{X} \subseteq \mathbb{Z}^n$ is the bounded, discrete feasible region. We compare different feasible solutions based on the principle of *Pareto dominance*. For any two feasible solutions $\mathbf{x}^1, \mathbf{x}^2 \in \mathcal{X}$, the solution $\mathbf{x}^1$ is said to Pareto dominate $\mathbf{x}^2$, denoted by $\mathbf{f}(\mathbf{x}^1) \prec \mathbf{f}(\mathbf{x}^2)$, if $f_i(\mathbf{x}^1) \leq f_i(\mathbf{x}^2)$ for all $i \in \{1, \dots, m\}$, and there exists at least one index $j \in \{1, \dots, m\}$ strictly satisfying $f_j(\mathbf{x}^1) < f_j(\mathbf{x}^2)$. An objective vector $\mathbf{y}^* = \mathbf{f}(\mathbf{x}^*)$ associated with a feasible solution $\mathbf{x}^* \in \mathcal{X}$ is defined as a *nondominated point* if there exists no other feasible solution $\mathbf{x} \in \mathcal{X}$ such that $\mathbf{f}(\mathbf{x}) \prec \mathbf{y}^*$. The collection of all such nondominated points across the entire feasible region constitutes the *Pareto frontier* (or non-dominated frontier) of the MODO problem. Mathematically, this frontier is expressed as $\mathcal{Y}_N = \{\mathbf{f}(\mathbf{x}) \mid \mathbf{x} \in \mathcal{X}, \nexists \mathbf{x}' \in \mathcal{X} \text{ such that } \mathbf{f}(\mathbf{x}') \prec \mathbf{f}(\mathbf{x})\}$. The fundamental aim in exact multiobjective optimization is to completely enumerate this Pareto frontier, providing decision-makers with the exact trade-offs among the competing criteria.

2.2 Decision Diagrams for Multiobjective Discrete Optimization

A DD is a directed layered acyclic graph $\mathcal{D} = (\mathcal{N}, \mathcal{A})$ defined by a node set $\mathcal{N}$ and an arc set $\mathcal{A}$. The node set is partitioned into $n + 1$ strictly sequential layers, $\mathcal{N} = \mathcal{L}_1 \cup \dots \cup \mathcal{L}_{n+1}$, corresponding to the sequence of the n decision variables. The first layer $\mathcal{L}_1$ contains only the *root node* $\mathbf{r}$, and the final layer $\mathcal{L}_{n+1}$ contains only the *terminal node* $\mathbf{t}$.

The arcs in $\mathcal{A}$ are directed from nodes in layer $\mathcal{L}_j$ to nodes in layer $\mathcal{L}_{j+1}$ for $j \in \{1, \dots, n\}$. Every arc $a \in \mathcal{A}$ originating from layer $\mathcal{L}_j$ possesses two key attributes: an *assignment value* $d(a) \in \mathbb{Z}$, which dictates the value assigned to the decision variable x_j , and an *objective weight* $\mathbf{v}(a) \in \mathbb{R}^m$, which represents the transition's contribution to the m -dimensional objective function. A path $p = (a_1, \dots, a_n)$ from the root $\mathbf{r}$ to the terminal node $\mathbf{t}$ encodes a complete decision vector $\mathbf{x}(p) = (d(a_1), \dots, d(a_n))^\top \in \mathbb{Z}^n$, with an

Algorithm 1 Coupled Pareto Frontier Enumeration

Require: Exact decision diagram $\mathcal{D} = (\mathcal{N}, \mathcal{A})$ with layers $\mathcal{L}_1, \dots, \mathcal{L}_{n+1}$

```

1: procedure FILTERDOMINATED( $\mathcal{Y}$ )  $\triangleright \mathcal{O}(|\mathcal{Y}|^2)$ 
2:   return  $\mathcal{Y} \leftarrow \{\mathbf{y} \in \mathcal{Y} \mid \nexists \mathbf{y}' \in \mathcal{Y} \text{ such that } \mathbf{y}' \prec \mathbf{y}\}$ 
3: end procedure
4: procedure EXPANDTOPDOWN( $j, \mathcal{Y}^\downarrow$ )
5:   for each node  $u \in \mathcal{L}_j$  do
6:     for each arc  $a = (u, v)$  originating from  $u$  do
7:        $\mathcal{Y}^\downarrow(v) \leftarrow \mathcal{Y}^\downarrow(v) \cup (\mathcal{Y}^\downarrow(u) \oplus \{\mathbf{v}(a)\})$ 
8:        $\mathcal{Y}^\downarrow(v) \leftarrow \text{FILTERDOMINATED}(\mathcal{Y}^\downarrow(v))$ 
9:     end for
10:   end for
11:   return  $\mathcal{Y}^\downarrow$ 
12: end procedure
13: procedure EXPANDBOTTOMUP( $j, \mathcal{Y}^\uparrow$ )
14:   for each node  $v \in \mathcal{L}_j$  do
15:     for each arc  $a = (u, v)$  entering  $v$  from  $u \in \mathcal{L}_{j-1}$  do
16:        $\mathcal{Y}^\uparrow(u) \leftarrow \mathcal{Y}^\uparrow(u) \cup (\mathcal{Y}^\uparrow(v) \oplus \{\mathbf{v}(a)\})$ 
17:        $\mathcal{Y}^\uparrow(u) \leftarrow \text{FILTERDOMINATED}(\mathcal{Y}^\uparrow(u))$ 
18:     end for
19:   end for
20:   return  $\mathcal{Y}^\uparrow$ 
21: end procedure
22:  $\mathcal{Y}^\downarrow(u) \leftarrow \emptyset, \mathcal{Y}^\uparrow(u) \leftarrow \emptyset$  for all  $u \in \mathcal{N}$ 
23:  $\mathcal{Y}^\downarrow(\mathbf{r}) \leftarrow \{\mathbf{0}\}, \mathcal{Y}^\uparrow(\mathbf{t}) \leftarrow \{\mathbf{0}\}$   $\triangleright \mathbf{r}$  is root,  $\mathbf{t}$  is terminal
24:  $i_{\text{td}} \leftarrow 1, i_{\text{bu}} \leftarrow n + 1$ 
25: while  $i_{\text{td}} \neq i_{\text{bu}}$  do
26:   Dynamically decide whether to expand top-down or bottom-up
27:   if expanding top-down then
28:     EXPANDTOPDOWN( $i_{\text{td}}, \mathcal{Y}^\downarrow$ ),  $i_{\text{td}} \leftarrow i_{\text{td}} + 1$ 
29:   else
30:     EXPANDBOTTOMUP( $i_{\text{bu}}, \mathcal{Y}^\uparrow$ ),  $i_{\text{bu}} \leftarrow i_{\text{bu}} - 1$ 
31:   end if
32: end while
33:  $c \leftarrow i_{\text{td}}, \mathcal{Y}_N \leftarrow \emptyset$ 
34: for each node  $u \in \mathcal{L}_c$  do
35:    $\mathcal{Y}(u) \leftarrow \mathcal{Y}^\downarrow(u) \oplus \mathcal{Y}^\uparrow(u)$   $\triangleright \mathcal{O}(|\mathcal{Y}^\downarrow(u)| |\mathcal{Y}^\uparrow(u)|)$ 
36: end for
37: for each node  $u \in \mathcal{L}_c$  do
38:    $\mathcal{Y}_N \leftarrow \text{FILTERDOMINATED}(\mathcal{Y}_N \cup \mathcal{Y}(u))$ 
39: end for
40: return  $\mathcal{Y}_N$ 

```

associated objective vector obtained by accumulating the arc weights along the path $\mathbf{v}(p) = \sum_{i=1}^n \mathbf{v}(a_i)$.

Let $\mathcal{X}_{\mathcal{D}} = \{\mathbf{x}(p) \mid p \text{ is an } \mathbf{r} \text{--} \mathbf{t} \text{ path in } \mathcal{D}\}$ denote the set of solutions encoded by the diagram. A DD is said to be *exact* for a given problem if it represents the feasible region ($\mathcal{X}_{\mathcal{D}} = \mathcal{X}$) and the path evaluations correctly compute the objective function ($\mathbf{v}(p) = \mathbf{f}(\mathbf{x}(p))$ for all paths). In this case, the image of the encoded solutions in the objective space is $\mathcal{Y}_{\mathcal{D}} = \{\mathbf{v}(p) \mid p \text{ is an } \mathbf{r} \text{--} \mathbf{t} \text{ path}\}$. The nondominated vectors extracted from $\mathcal{Y}_{\mathcal{D}}$ constitute the exact Pareto frontier $\mathcal{Y}_N$ of the original MODO problem.

To extract this exact Pareto frontier $\mathcal{Y}_N$ from an exact decision diagram $\mathcal{D}$, we can employ the *coupled enumeration scheme*, which dynamically interleaves top-down and bottom-up layer expansion, as detailed in Algorithm 1. This dynamic layer cutset methodology mitigates intermediate label explosion. Initially, a top-down layer pointer i_{td} is set to layer 1 and a bottom-up pointer i_{bu} is set to layer $n + 1$. At each step, the scheme dynamically decides whether to expand the top-down frontier $\mathcal{Y}^\downarrow$ to layer $i_{td} + 1$ or the bottom-up frontier $\mathcal{Y}^\uparrow$ to layer $i_{bu} - 1$. This process continues until the two frontiers meet at a *cutset* layer c (i.e., $i_{td} = i_{bu}$). Finally, for each node $u \in \mathcal{L}_c$, the algorithm computes the convolution of its top-down and bottom-up labels (Algorithm 1, line 35), taking the pairwise sum of these vectors. The merging of these combined sets and the repeated application of a dominance filter produce the exact Pareto frontier $\mathcal{Y}_N$.

A special case of the coupled enumeration scheme is the *top-down enumeration scheme*, in which the bottom-up layer pointer i_{bu} is fixed to the layer $n + 1$. As such, only operations involving ‘ExpandTopDown’ are executed, and the cutset layer naturally occurs at $c = n + 1$. In this case, upon completion of the final layer processing, the label set $\mathcal{Y}^\downarrow(\mathbf{t})$ at the terminal node directly corresponds to the exact Pareto frontier $\mathcal{Y}_N$. This approach is outlined in Algorithm 2. Note that we did not perform convolution or repeated dominance filtering (Algorithm 1, lines 33 to 37) in this approach.

Algorithm 2 Top-Down Pareto Frontier Enumeration

Require: Exact decision diagram $\mathcal{D} = (\mathcal{N}, \mathcal{A})$ with layers $\mathcal{L}_1, \dots, \mathcal{L}_{n+1}$

- 1: $\mathcal{Y}^\downarrow(u) \leftarrow \emptyset$ for all $u \in \mathcal{N}$
- 2: $\mathcal{Y}^\downarrow(\mathbf{r}) \leftarrow \{\mathbf{0}\}$ $\triangleright \mathbf{r}$ is the root node in $\mathcal{L}_1$, $\mathbf{0} \in \mathbb{R}^m$
- 3: **for** $j = 1$ **to** n **do**
- 4: EXPANDTOPDOWN($j, \mathcal{Y}^\downarrow$)
- 5: **end for**
- 6: **return** $\mathcal{Y}^\downarrow(\mathbf{t})$ $\triangleright \mathbf{t}$ is the terminal node in $\mathcal{L}_{n+1}$

2.3 Graphics Processing Unit

GPUs are throughput-oriented processors that achieve high performance by executing many lightweight threads in parallel. In contrast, CPUs comprise a small number of complex cores optimized for low-latency execution and sophisticated control flow. This architectural difference makes GPUs particularly effective for data-parallel workloads in which the same operation is applied independently across many inputs (?). Figure 1 illustrates the GPU execution and memory hierarchy. Computation is launched as a *kernel*, which executes as a large collection of *threads*. Threads are grouped into *thread blocks*, and all blocks in a kernel launch form a *grid* (?). The GPU schedules thread blocks onto *Streaming Multiprocessors* (SMs), where each SM can host multiple blocks concurrently and executes threads in a *Single Instruction Multiple Thread* (SIMT) manner, where threads follow the same instruction stream while operating on different data.

The memory system is also hierarchical. Threads within an SM have access to fast on-chip storage, including regis-

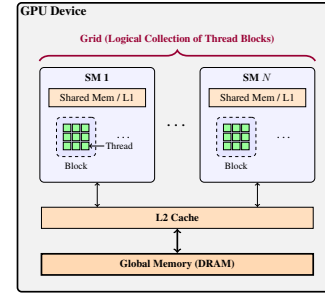


Figure 1: Schematic representation of a GPU architecture.

ters and shared memory, which enables efficient intra-block cooperation and synchronization. SMs connect to a larger, device-wide cache (L2) and ultimately to high-capacity but high-latency global memory (DRAM). Effective GPU implementations therefore aim to expose ample parallelism while limiting expensive global-memory traffic (?). This execution model is well suited for workloads dominated by large numbers of independent arithmetic operations and comparisons. In particular, pairwise dominance checks required during PF enumeration in Algorithm 2 are natural candidates for GPU acceleration.

3 Parallel Decision Diagram-based Pareto Frontier Enumeration

This section describes the methodology for leveraging parallelism during the PF enumeration. The different parallelization schemes are introduced in Section 3.1 and their implementation details are provided in Section 3.2.

3.1 Parallelization Schemes

Single thread per node In this scheme, we exploit the inherent independence of nodes within the same layer to achieve parallelism. Specifically, the processing of any node $v \in \mathcal{L}_{j+1}$ relies exclusively on the finalized objective vectors of its predecessor nodes $u \in \mathcal{L}_j$. Because the operations performed on different destination nodes do not mutate or conflict with one another’s memory space, we can process multiple nodes concurrently by assigning a single, dedicated execution thread to each node.

For instance, in the coupled enumeration scheme (Algorithm 1), the execution of EXPANDTOPDOWN (lines 5-10) and EXPANDBOTTOMUP (lines 14-19) can be parallelized over the outer loop of destination nodes. Furthermore, at the cutset layer $\mathcal{L}_c$, the convolution of the top-down and bottom-up frontiers (lines 34-36) can be parallelized across individual nodes since each convolution operates strictly on local label sets $\mathcal{Y}^\downarrow(u)$ and $\mathcal{Y}^\uparrow(u)$ associated with node u . By mapping a single thread per node, this scheme circumvents data races without the need for expensive thread synchronization mechanisms like locks.

Multiple threads per node While assigning a single thread per node is straightforward, the overall enumeration can be further accelerated by utilizing multiple threads to

process a single node collaboratively. However, a naive implementation of such a scheme presents significant synchronization hazards. Consider the EXPANDTOPDOWN procedure, where multiple incoming arcs (u, v) contribute to the same destination node v . A simplistic approach to multi-threading this step would be to assign a separate thread to each incoming arc. Each thread would expand the labels of the parent node of its assigned arc and attempt to merge and filter the generated candidates directly into the destination node's shared set, $\mathcal{Y}^\downarrow(v)$. This introduces the following critical synchronization issues.

1. **Data Races and Loss:** If threads attempt to update the shared label set concurrently without locks, memory corruption will occur, leading to the unpredictable loss of valid Pareto-optimal candidates.
2. **Synchronization Bottlenecks:** If a lock is applied to the shared label set $\mathcal{Y}^\downarrow(v)$ to prevent data races, the execution essentially serializes, negating the benefits of multi-threading.

To effectively leverage multiple cores per node without succumbing to locking bottlenecks, we can trade off memory for execution speed. Rather than merging candidates into a single shared set on the fly, we can decouple the expansion phase from the filtering phase. First, threads can safely evaluate the addition for all arcs in parallel, scattering the resulting candidate labels into disjoint, pre-allocated memory segments. Once all candidates for the node are materialized in memory, a pool of threads can cooperatively perform the computationally dominant $\mathcal{O}(|\mathcal{Y}|^2)$ dominance filtering operation over this complete set of candidates, allowing multiple threads to actively prune suboptimal vectors simultaneously without locks.

Furthermore, assigning a *fixed* number of threads to collaboratively process each node is suboptimal if the size of the intermediate Pareto frontiers $\mathcal{Y}^\downarrow(v)$ varies drastically across nodes within a given layer. If a static number of threads are allocated per node, the nodes with massive sets of incoming candidates will still bottleneck the overall layer progression, while the thread groups assigned to nodes with few candidates will prematurely finish and sit idle. Thus, an optimal multi-threaded strategy must dynamically allocate a variable number of threads to each node proportional to the size of its candidate label set to ensure an evenly balanced workload. In practice, our CPU implementation realizes the *single thread per node* scheme, while the GPU architecture allows us to efficiently realize the *multiple threads per node* scheme.

3.2 Implementation

Central Processing Unit The CPU implementation directly realizes the single thread per node parallelization scheme. At each layer j , the loop over destination nodes $v \in \mathcal{L}_{j+1}$ is parallelized with a *dynamic work-stealing* schedule: since the number of incoming candidate labels per node varies widely across a layer, a static round-robin assignment would leave many cores idle while others process large frontier sets. Each assigned core processes all incoming arcs (u, v) for its node sequentially, accumulating the

Algorithm 3 GPU-Accelerated Top-Down Pareto Frontier Enumeration

Require: DD $\mathcal{D}$ with layers $\mathcal{L}_1, \dots, \mathcal{L}_{n+1}$

- 1: **on CPU:** For every arc $a \in \mathcal{A}$, pack source-node index $b(a)$ and weight vector $\mathbf{v}(a)$ into flat arrays; for every node v , pack the incoming-arc offset array ϕ_v^a ; transfer all to GPU $\triangleright$ One-time
 - 2: **on CPU:** Initialize $\mathcal{Y}^\downarrow(\mathbf{r}) \leftarrow \{\mathbf{0}\}$ in GPU memory
 - 3: **for** $j = 1$ **to** n [**CPU outer loop**] **do**
 - 4: $\kappa, \phi^c, \Gamma \leftarrow \text{COMPUTEARCOFFSETS}(\mathcal{Y}^\downarrow, b, \phi^a)$
 - 5: $\beta \leftarrow \text{EXPANDCANDIDATES}(\mathcal{Y}^\downarrow, b, \mathbf{v}, \phi^c, \kappa)$
 - 6: $\sigma \leftarrow \text{FILTERDOMINATED}(\beta, \Gamma)$
 - 7: $\mathcal{Y}^\downarrow \leftarrow \text{COMPACTSURVIVORS}(\beta, \sigma)$
 - 8: Swap buffer pointers (no data copy)
 - 9: **end for**
 - 10: **on CPU:** Copy $\mathcal{Y}^\downarrow(\mathbf{t})$ from GPU to host; **return** $\mathcal{Y}^\downarrow(\mathbf{t})$
-

Minkowski sums $\mathcal{Y}^\downarrow(u) \oplus \{\mathbf{v}(a)\}$ and applying the $\mathcal{O}(|\mathcal{Y}|^2)$ dominance filter entirely in thread-local memory, requiring no locks or synchronization. However, a core assigned to a node with a very large local frontier cannot offload work to idle cores, throttling the entire layer's throughput. This load-imbalance limitation motivates the GPU design below.

Graphics Processing Unit The GPU is well-suited to realize the multiple threads per node scheme: its massively parallel architecture executes thousands of lightweight threads concurrently, making it ideal for absorbing the data-parallelism in candidate generation and dominance filtering. Algorithm 3 presents the GPU-accelerated top-down enumeration, which decomposes each layer's processing into four data-parallel GPU passes that call the subroutines in Algorithms 4–7.

Before entering the layer loop, the CPU serializes the DD's arc connectivity into contiguous flat arrays—the source-node index $b(a)$, the m -dimensional weight vector $\mathbf{v}(a)$ for every arc a , and the per-node incoming-arc offset array ϕ_v^a —and transfers them to GPU global memory once (line 1), amortizing the packing cost across all layers. The outer loop executes on the CPU because each layer depends on the previous layer's filtered results. Within each layer, four GPU passes are executed.

(1) Arc-level offset computation (Algorithm 4): for each arc $a = (u, v)$, one GPU thread reads the source node's frontier size $|\mathcal{Y}^\downarrow(u)|$, yielding a per-arc candidate count κ_a . An exclusive prefix sum over these counts produces per-arc buffer offsets ϕ_a^c , which determine where each arc's candidates will be written in the flat buffer β . The per-destination-node candidate total Γ_v is derived from the difference between the edge offsets of the last and first incoming arcs of v .

(2) Candidate expansion (Algorithm 5): the flat buffer β is populated with shifted labels. Each arc a is assigned to a *warp*—a fixed group of 32 GPU threads that execute the

⁰For problems with tractable state-space structure (e.g., knapsack), a problem-specific dominance filter is additionally applied on the GPU after this step.

Algorithm 4 GPU Subroutine: Compute Arc Offsets

Require: Frontier $\mathcal{Y}^\downarrow$; source-node array $b[]$; incoming-arc offsets $\phi^a[]$

- 1: **procedure** COMPUTEARCSETS($\mathcal{Y}^\downarrow, b, \phi^a$)
- 2: $M \leftarrow |\mathcal{A}_{\mathcal{L}_j}|$ $\triangleright$ Number of arcs in layer j
- 3: **for all** arcs $a = 0$ **to** $M - 1$ **in parallel on GPU do**
 $\triangleright$ One thread per arc
- 4: $\kappa_a \leftarrow |\mathcal{Y}^\downarrow(b(a))|$
- 5: **end for**
- 6: $\phi_0^c \leftarrow 0$
- 7: **for** $a = 1$ **to** M **in parallel on GPU do**
- 8: $\phi_a^c \leftarrow \phi_{a-1}^c + \kappa_{a-1}$
- 9: **end for**
- 10: **for all** nodes $v \in \mathcal{L}_{j+1}$ **in parallel on GPU do**
- 11: $a_{\text{first}} \leftarrow \phi_v^a$
- 12: $a_{\text{last}} \leftarrow \phi_{v+1}^a$
- 13: $\Gamma_v \leftarrow \phi_{a_{\text{last}}}^c - \phi_{a_{\text{first}}}^c$
- 14: **end for**
- 15: **return** κ, ϕ^c, Γ
- 16: **end procedure**

Algorithm 5 GPU Subroutine: Expand Candidates into Flat Buffer

Require: Frontier $\mathcal{Y}^\downarrow$; source array $b[]$; arc weights $\mathbf{v}(\cdot)$; offsets $\phi^c[]$; counts $\kappa[]$

- 1: **procedure** EXPANDCANDIDATES($\mathcal{Y}^\downarrow, b, \mathbf{v}, \phi^c, \kappa$)
- 2: Allocate flat buffer β of size $\sum_a \kappa_a$
- 3: **for all** arcs a **in parallel on GPU do**
- 4: $\triangleright$ One warp (32 threads) per arc
- 5: **for all** $k = 0, 1, \dots, \kappa_a - 1$ **distributed across warp threads do**
- 6: $\beta[\phi_a^c + k] \leftarrow \mathcal{Y}^\downarrow(b(a))[k] + \mathbf{v}(a)$
- 7: **end for**
- 8: **end for**
- 9: **return** β
- 10: **end procedure**

same instruction simultaneously, each on different data. The threads within a warp cooperatively iterate over the source node's labels, with each thread handling one label at a time. Since the prefix sum in Step 1 guarantees that each arc's output segment $[\phi_a^c, \phi_a^c + \kappa_a)$ is non-overlapping, threads for different arcs write to disjoint memory regions without synchronization.

(3) **Load-balanced dominance filtering** (Algorithm 6): each destination node v receives $K_v = \lceil \Gamma_v / B \rceil$ thread blocks (where B is the block size). A prefix sum over K_v yields a block-offset array ϕ^k that maps each block to its target node. A tightly packed grid of $\sum_v K_v$ blocks is launched; each block locates its target node via binary search, ensuring no blocks sit idle.

Within each thread block, two levels of the memory hierarchy are exploited. Each thread loads its own candidate $\mathbf{y}_i$ from global GPU memory (high-latency DRAM) once into a *per-thread private register* (line 10), retaining it for all tile comparisons without re-reading global memory. The

Algorithm 6 GPU Dominance Filtering: Dynamic 1D Load-Balanced Scheduling

Require: Flat candidate buffer β ; per-node candidate counts $\Gamma[]$; block size B

- 1: **procedure** FILTERDOMINATED(β, Γ)
- 2: $K[v] \leftarrow \lceil \Gamma[v] / B \rceil$ for all v
- 3: $\phi_0^k \leftarrow 0$
- 4: **for** $v = 1$ **to** $|\mathcal{L}_{j+1}|$ **in parallel on GPU do**
- 5: $\phi_v^k \leftarrow \phi_{v-1}^k + K[v - 1]$
- 6: **end for**
- 7: $\sigma_i \leftarrow 1$ for all candidates i
- 8: **for all** thread blocks $q = 0, \dots, \sum_v K[v] - 1$ **in parallel on GPU do**
- 9: Binary search ϕ^k to find node v s.t. $\phi_v^k \leq q < \phi_{v+1}^k$
- 10: $\tau \leftarrow q - \phi_v^k$ $\triangleright$ Tile index within node v
- 11: $[b, e) \leftarrow$ candidate range of node v in β
- 12: $i \leftarrow b + \tau \cdot B +$ thread index within block
- 13: **Load** $\mathbf{y}_i \leftarrow \beta[i]$ into per-thread register (if $i < e$) $\triangleright$ Reused across tiles
- 14: $\text{dominated} \leftarrow \text{false}$
- 15: **for** tile start $s = b$ **to** $e - 1$, **step** B **do**
- 16: **Barrier:** wait for all threads in block
- 17: Each thread k loads $\beta[s+k]$ into **shared memory** slot k (if $s+k < e$)
- 18: **Barrier:** ensure tile is fully visible
- 19: **if** $i < e$ **and not** dominated **then**
- 20: **for** $j = 0$ **to** $\min(B, e-s)-1$, **skip** $j = i - b - \tau B$ **do**
- 21: **if** $\text{shared}[j] \prec \mathbf{y}_i$ **then**
 $\text{dominated} \leftarrow \text{true}; \text{break}$
- 22: **end if**
- 23: **end for** $\triangleright$ Early exit within tile
- 24: **end if**
- 25: **if** dominated **then break**
- 26: **end if** $\triangleright$ Early exit across tiles
- 27: **end for**
- 28: **if** $i < e$ **and** dominated **then** $\sigma_i \leftarrow 0$
- 29: **end if**
- 30: **end for**
- 31: **return** σ
- 32: **end procedure**

Algorithm 7 GPU Subroutine: Compact Survivors

Require: Candidate buffer β of size S ; binary survivor flags $\sigma[]$

```

1: procedure COMPACTSURVIVORS( $\beta, \sigma$ )
2:    $\rho_0 \leftarrow 0$ 
3:   for  $i = 1$  to  $S$  do  $\triangleright$  Parallel prefix sum on GPU
4:      $\rho_i \leftarrow \rho_{i-1} + \sigma_{i-1}$ 
5:   end for
6:   for all candidates  $i = 0$  to  $S-1$  in parallel on GPU
7:     do  $\triangleright$  One thread per candidate
8:       if  $\sigma_i = 1$  then
9:          $\mathcal{Y}^\downarrow(v)[\rho_i] \leftarrow \beta[i] \triangleright v$ : destination node of  $i$ 
10:       end if
11:   return  $\mathcal{Y}^\downarrow$ 
12: end procedure
```

B block members then cooperatively load a tile of B reference candidates into *shared memory* (fast on-chip scratchpad visible to all threads in the block), one reference per thread (lines 13–14). Two barrier synchronizations bracket this load so all threads see a consistent tile. Each thread checks $\mathbf{y}_i$ against all B references; a dominance detection immediately breaks both the within-tile and across-tile loops (lines 17–19), avoiding redundant comparisons.

(4) Survivor compaction (Algorithm 7): the dominance filter produces a binary flag $\sigma_i \in \{0, 1\}$ for each candidate i , where $\sigma_i = 1$ indicates that candidate i was not Pareto dominated. An exclusive prefix sum over these flags computes each survivor’s compacted output position ρ_i . A final scatter pass writes only the survivors into the per-node label sets $\mathcal{Y}^\downarrow(v)$.

GPU-Accelerated Coupled Enumeration While top-down expansion is highly parallel, memory consumption can scale rapidly for intermediate layers with large widths. The coupled enumeration scheme (Algorithm 1) mitigates this by dynamically progressing from both directions to meet at a cutset layer $\mathcal{L}_c$.

Algorithm 8 presents the GPU-accelerated coupled enumeration orchestrator. Before entering the selection loop, the CPU serializes the entire DD’s structure and transfers it to the GPU once (line 1). The top-down and bottom-up frontiers are initialized at the root and terminal nodes respectively in GPU memory (line 2). The dynamic layer selection loop then executes on the CPU, dynamically choosing to expand either the top-down layer i_{td} or the bottom-up layer i_{bu} based on the computed heuristic expansion scores s_{td} and s_{bu} (lines 5–13). Each layer expansion is performed entirely on the GPU utilizing the data-parallel routines detailed in Algorithms 4–7.

Once the top-down and bottom-up frontiers meet at the cutset layer $\mathcal{L}_c$, the final convolving and merging step is delegated to the GPU subroutine CUTSETCOUPLING (Algorithm 9). This subroutine filters for active nodes $\mathcal{V}_c^* \subseteq \mathcal{L}_c$ that possess non-empty frontiers in both directions (line 1), since nodes lacking either cannot produce feasible complete paths. To maximize GPU execution efficiency and prevent

Algorithm 8 GPU-Accelerated Coupled Pareto Frontier Enumeration

Require: DD $\mathcal{D}$ with layers $\mathcal{L}_1, \dots, \mathcal{L}_{n+1}$

```

1: on CPU: Pack all layers into flat GPU arrays; transfer to GPU  $\triangleright$  One-time
2: on CPU: Initialize  $\mathcal{Y}^\downarrow(\mathbf{r}) \leftarrow \{\mathbf{0}\}$  and  $\mathcal{Y}^\uparrow(\mathbf{t}) \leftarrow \{\mathbf{0}\}$  in GPU memory
3:  $i_{td} \leftarrow 1$ ;  $i_{bu} \leftarrow n + 1$ 
4:  $s_{td} \leftarrow 0$ ;  $s_{bu} \leftarrow 0$ 
5: while  $i_{td} \neq i_{bu}$  do
6:   if  $s_{td} \leq s_{bu}$  then  $\triangleright$  Expand top-down
7:      $i_{td} \leftarrow i_{td} + 1$ 
8:      $\mathcal{Y}^\downarrow \leftarrow \text{EXPANDLAYERTOPDOWN}(\mathcal{Y}^\downarrow, i_{td})$ 
9:      $s_{td} \leftarrow \text{COMPUTESCORE}(\mathcal{Y}^\downarrow, i_{td})$ 
10:  else  $\triangleright$  Expand bottom-up
11:     $i_{bu} \leftarrow i_{bu} - 1$ 
12:     $\mathcal{Y}^\uparrow \leftarrow \text{EXPANDLAYERBOTTOMUP}(\mathcal{Y}^\uparrow, i_{bu})$ 
13:     $s_{bu} \leftarrow 1.5 \times \text{COMPUTESCORE}(\mathcal{Y}^\uparrow, i_{bu})$ 
14:  end if
15: end while
16:  $c \leftarrow i_{td}$   $\triangleright$  Cutset layer
17:  $\mathcal{Y}_N \leftarrow \text{CUTSETCOUPLING}(\mathcal{Y}^\downarrow, \mathcal{Y}^\uparrow, \mathcal{L}_c)$ 
18: return  $\mathcal{Y}_N$ 
```

warp divergence, the active nodes are sorted in descending order of their total frontier sizes $\text{score}(v) = \kappa_v^\downarrow + \kappa_v^\uparrow$. They are then processed in load-balanced, memory-bounded batches $\mathcal{B}$ s.t. the total number of Cartesian product candidates generated per batch does not exceed $B_{\max}$ (default 625,000).

For each batch $\mathcal{B}$, a custom GPU kernel materializes the Minkowski sum candidates β (Algorithm 10). Since convolving clean frontiers can generate redundant duplicates or dominated points, each batch undergoes a three-stage filtering pipeline on the GPU: (1) filtering against the running global frontier $\mathcal{Y}^{\text{global}}$ (line 9) to prune candidates dominated by already found solutions, (2) local self-pruning (line 11) using the 1D dominance filter to remove intra-batch dominated points, and (3) mutual merging (line 13) which filters the running global frontier against the batch survivors, and then concatenates the remaining candidates.

4 Computational Experiments

We evaluate our proposed methods on MOKP, MOSPP and MOTSP. The test instances are sourced from (?). For each instance size configuration, it consists of 10 instances. We report sizes for which coupled enumeration schemes (Couple or Couple⁺) required at least 5 seconds of total computation time. We also exclude sizes for which the coupled enumeration scheme was unable to solve any instances. All experiments were conducted on the Nibi cluster, equipped with an Intel Xeon 6972P CPU @ 2.4 GHz and an Nvidia H100 GPU, with 20 GB of RAM and a 1-hour time limit per instance. The parallel PF enumeration was implemented in C++14, utilizing OpenMP 4.5 for CPU parallelism and NVCC 12.6 for GPU acceleration.

We compare the runtime performance of the serial top-

Table 1: MOKP Pareto Enumeration Runtime Comparison Across TD/Coup CPU and GPU Methods

N	K	Method	Total	Enum.	(M, T)	N	K	Method	Total	Enum.	(M, T)
40	6	TD ⁺	33	32	(0, 0)	40	7	TD ⁺	79	79	(0, 0)
40	6	TD ⁺ -4	22	22	(0, 0)	40	7	TD ⁺ -4	54	54	(0, 0)
40	6	TD ⁺ -8	20	20	(0, 0)	40	7	TD ⁺ -8	52	52	(0, 0)
40	6	TD ⁺ -16	20	20	(0, 0)	40	7	TD ⁺ -16	49	49	(0, 0)
40	6	Coup ⁺	10	10	(0, 0)	40	7	Coup ⁺	32	32	(0, 0)
40	6	Coup ⁺ -4	5	5	(0, 0)	40	7	Coup ⁺ -4	15	15	(0, 0)
40	6	Coup ⁺ -8	5	5	(0, 0)	40	7	Coup ⁺ -8	13	13	(0, 0)
40	6	Coup ⁺ -16	5	4	(0, 0)	40	7	Coup ⁺ -16	11	11	(0, 0)
40	6	TD ⁺ -GPU	5	3	(0, 0)	40	7	TD ⁺ -GPU	7	5	(0, 0)
50	5	TD ⁺	82	82	(0, 0)	50	6	TD ⁺	440	439	(0, 1)
50	5	TD ⁺ -4	53	53	(0, 0)	50	6	TD ⁺ -4	306	306	(0, 1)
50	5	TD ⁺ -8	49	48	(0, 0)	50	6	TD ⁺ -8	283	283	(0, 1)
50	5	TD ⁺ -16	47	47	(0, 0)	50	6	TD ⁺ -16	275	275	(0, 1)
50	5	Coup ⁺	19	19	(0, 0)	50	6	Coup ⁺	185	185	(0, 0)
50	5	Coup ⁺ -4	10	9	(0, 0)	50	6	Coup ⁺ -4	79	78	(0, 0)
50	5	Coup ⁺ -8	8	8	(0, 0)	50	6	Coup ⁺ -8	62	62	(0, 0)
50	5	Coup ⁺ -16	7	7	(0, 0)	50	6	Coup ⁺ -16	55	55	(0, 0)
50	5	TD ⁺ -GPU	6	4	(0, 0)	50	6	TD ⁺ -GPU	49	46	(0, 0)
50	7	TD ⁺	1559	1558	(0, 1)	60	4	TD ⁺	112	112	(0, 0)
50	7	TD ⁺ -4	1110	1110	(0, 1)	60	4	TD ⁺ -4	77	76	(0, 0)
50	7	TD ⁺ -8	1044	1043	(0, 1)	60	4	TD ⁺ -8	71	70	(0, 0)
50	7	TD ⁺ -16	998	998	(0, 1)	60	4	TD ⁺ -16	69	68	(0, 0)
50	7	Coup ⁺	507	507	(0, 0)	60	4	Coup ⁺	16	15	(0, 0)
50	7	Coup ⁺ -4	229	229	(0, 0)	60	4	Coup ⁺ -4	8	8	(0, 0)
50	7	Coup ⁺ -8	185	185	(0, 0)	60	4	Coup ⁺ -8	7	6	(0, 0)
50	7	Coup ⁺ -16	168	168	(0, 0)	60	4	Coup ⁺ -16	6	6	(0, 0)
50	7	TD ⁺ -GPU	85	83	(0, 0)	60	4	TD ⁺ -GPU	5	3	(0, 0)

down (TD) and coupled PF enumeration (Coup) methods against various parallel implementations. To evaluate CPU-based parallelization, we append a suffix -T to denote multi-threaded versions, where T specifies the number of allocated threads (e.g., TD-8 indicates the top-down method utilizing 8 threads). TD-GPU represents our GPU-accelerated implementation of the top-down method. Finally, method names appended with a plus sign (TD⁺ and Coup⁺) indicate that state-based label filtering was applied after performing Pareto dominance filtering at each layer.

We report the overall wall-clock time (“Total”) and the time spent specifically on enumeration (“Enum.”), in seconds. For CPU-based methods, the Enum. time excludes the time required to build the decision diagram (DD), whereas for the GPU variant, it includes the time spent running GPU kernels. Lastly, the tuple “(M, T)” reports the number of instances that failed due to out-of-memory errors (M) and time-out limits (T).

4.1 Results

The runtime performance for PF enumeration for MOKP, MOSPP and MOTSP is reported in Table 1, Table 3 and Table 4, respectively. Our experiments demonstrate that parallelism consistently improves enumeration times. Across all problem classes and sizes, either a multi-core CPU variant or the GPU-accelerated variant achieves the fastest Total runtime. Across instance sizes, the maximum speed-up achieved by a GPU variant is 7.49× for MOKP, 15.08× for MOSPP, and 13.60× for MOTSP relative to the Coup/Coup+ baseline. In comparison, the maximum speed-up achieved by the best Coup-T configuration is 3.36× for MOKP, 2.68× for MOSPP, and 5.75× for MOTSP relative to the Coup/Coup+.

The GPU variant (TD-GPU or TD⁺-GPU) emerges as the overall winner in the majority of cases, delivering the lowest execution times when it successfully completes. However, it is able to solve fewer instances overall due to out-of-memory errors. For example, in the MOTSP domain with $N = 15$ and $K = 6$, the GPU variant runs out of memory on all 10

Algorithm 9 GPU Subroutine: Cutset Coupling

Require: Cutset nodes $\mathcal{L}_c$; frontiers $\mathcal{Y}^\downarrow(v), \mathcal{Y}^\uparrow(v)$ and sizes $\kappa_v^\downarrow, \kappa_v^\uparrow$ for all $v \in \mathcal{L}_c$; candidate batch limit $B_{\max}$

- 1: $\mathcal{V}_c^* \leftarrow \{v \in \mathcal{L}_c \mid \kappa_v^\downarrow > 0 \text{ and } \kappa_v^\uparrow > 0\}$ $\triangleright$ Active nodes
- 2: $\text{score}(v) \leftarrow \kappa_v^\downarrow + \kappa_v^\uparrow$ for all $v \in \mathcal{V}_c^*$
- 3: Sort $\mathcal{V}_c^*$ in descending order of $\text{score}(v)$
- 4: $\mathcal{Y}^{\text{global}} \leftarrow \emptyset$ $\triangleright$ Running global frontier in GPU memory
- 5: Partition sorted $\mathcal{V}_c^*$ into batches $\mathcal{B}_1, \mathcal{B}_2, \dots$ s.t. $\sum_{v \in \mathcal{B}_k} \kappa_v^\downarrow \cdot \kappa_v^\uparrow \leq B_{\max}$
- 6: **for** each batch $\mathcal{B}$ **do**
- 7: $\beta \leftarrow \text{MATERIALIZEPRODUCTS}(\mathcal{Y}^\downarrow, \mathcal{Y}^\uparrow, \mathcal{B})$
- 8: **if** $\mathcal{Y}^{\text{global}} \neq \emptyset$ **then**
- 9: $\sigma \leftarrow \text{FILTERAGAINSTFRONTIER}(\beta, \mathcal{Y}^{\text{global}})$
- 10: $\beta \leftarrow \text{COMPACTSURVIVORS}(\beta, \sigma)$
- 11: **end if**
- 12: $\beta \leftarrow \text{SELFPRUNE}(\beta)$ $\triangleright$ Local 1D dominance check
- 13: $\mathcal{Y}^{\text{global}} \leftarrow \text{MERGEFRONTIERS}(\mathcal{Y}^{\text{global}}, \beta)$ $\triangleright$ Mutual pruning and concatenation
- 14: **end for**
- 15: **return** $\mathcal{Y}^{\text{global}}$

Algorithm 10 GPU Subroutine: Materialize Cutset Products

Require: Frontiers $\mathcal{Y}^\downarrow, \mathcal{Y}^\uparrow$; node offsets $\phi^\downarrow, \phi^\uparrow$ and sizes $\kappa_v^\downarrow, \kappa_v^\uparrow$ for nodes in batch $\mathcal{B}$; product offsets ϕ^p

- 1: **procedure** $\text{MATERIALIZEPRODUCTS}(\mathcal{Y}^\downarrow, \mathcal{Y}^\uparrow, \mathcal{B})$
- 2: Allocate flat candidate buffer β of size $\sum_{v \in \mathcal{B}} \kappa_v^\downarrow \cdot \kappa_v^\uparrow$
- 3: **for all** nodes $v \in \mathcal{B}$ **in parallel on GPU do** $\triangleright$ One thread block per node
- 4: $T \leftarrow \kappa_v^\downarrow \cdot \kappa_v^\uparrow$ $\triangleright$ Total candidates for node v
- 5: **for all** candidate indices $p = 0, \dots, T - 1$ **in parallel do**
- 6: $i \leftarrow \lfloor p / \kappa_v^\uparrow \rfloor$; $j \leftarrow p \pmod{\kappa_v^\uparrow}$
- 7: $y^\downarrow \leftarrow \mathcal{Y}^\downarrow(\phi_v^\downarrow + i)$; $y^\uparrow \leftarrow \mathcal{Y}^\uparrow(\phi_v^\uparrow + j)$
- 8: $\beta[\phi_v^p + p] \leftarrow y^\downarrow + y^\uparrow$
- 9: **end for**
- 10: **end for**
- 11: **return** β
- 12: **end procedure**

instances. Similar memory bottlenecks are observed for the MOSPP domain at $N = 200, K = 3$ ($M = 10$) and the MOKP domain at $N = 70, K = 7$ ($M = 7$). This limitation is expected, as the GPU implementation inherently generates and stores the full partial frontiers for all nodes in a given layer simultaneously to maximize parallel throughput.

When examining CPU-based parallelization, increasing the number of threads generally reduces the total enumeration time, though the speed-up is often sub-linear and exhibits diminishing returns. For instances with larger workloads, adding threads yields consistent improvements. For example, in the MOSPP with $N = 200, K = 4$, Coup-T runtimes decrease steadily from 72s (4 threads) to 60s (8 threads) down to 47s (16 threads). However, we frequently observe performance stagnation as thread counts increase from 8 to 16. Consider the MOTSP with $N = 15, K = 6$, where Coup-8 reduces the runtime to 785s (from 883s with 4 threads), but Coup-16 yields no further improvement, stalling at 785s. Similarly, for the MOKP with $N = 40, K = 6$, both parallel methods hit a scaling wall early, showing little to no speed-up beyond 4 or 8 threads. This might also be an artifact of the final merge (Algorithm 1, lines 37-39) being performed serially on the coupling layer.

Finally, when comparing the CPU-based parallel approaches, the parallel top-down enumeration (TD-T) is still slower than the parallel coupled enumeration (Coup-T). This aligns with our expectations regarding the efficiency of the coupled enumeration strategy.

5 Related Work

Parallel computing has long played an important role in accelerating discrete optimization algorithms (?). Early work primarily focused on multi-core CPUs and distributed systems, where task-level parallelism was exploited in search trees and decomposition algorithms. More recently, advances in accelerator hardware have enabled new forms of fine-grained parallelism, motivating the development of algorithms designed specifically for massively parallel architectures such as GPUs.

5.1 CPU-based Parallelism

Branch-and-Bound (BnB) is the dominant paradigm for exact combinatorial optimization, and its parallelization on shared-memory architectures has been studied extensively (??). (?) identified the “anomaly” phenomenon in parallel BnB, where superlinear speedups arise from lucky pruning in one thread preempting redundant work in others. Early work demonstrated that parallel BnB can significantly reduce solution times when the search space can be effectively partitioned (??). In LP, the most impactful application of shared-memory parallelism is in the factorization and solve phases of interior point methods (IPM), where the dominant sparse Cholesky solve is parallelized via multithreaded libraries such as PARDISO (?) and MA57 (?). For Simplex, the sequential coupling between pivot selection and basis factorization limits practical speedup despite efforts to parallelize the pricing step (?). A parallel DD-based BnB framework that distributes DD node exploration across multiple

processors, proposed in (?), demonstrated that it effectively scales with the number of CPU cores.

5.2 GPU-based Parallelism

Early GPU efforts targeted the Simplex method by offloading pricing and ratio test steps, with (?) and (?) demonstrating speedups in pivoting rules, though sequential basis updates remained a bottleneck. Attention has since shifted to IPM and first-order methods. A GPU-accelerated interior-point solver competitive with commercial CPU solvers on large-scale instances was introduced in (?). Building on (?), a series of GPU-native primal-dual hybrid gradient solvers (???) demonstrated that first-order methods can scale to massive LPs without factorization overhead. Accelerating Mixed-Integer Programming on GPUs presents additional challenges due to the irregular structure of the BnB tree. Most existing approaches adopt hybrid architectures in which the CPU manages the search tree while GPUs accelerate computational kernels. For instance, GPU-accelerated evaluation of node relaxations within a BnB framework was proposed in (?). Recent research has also investigated GPU-accelerated primal heuristics, such as feasibility pump, feasibility jump, and fix-and-propagate, to quickly produce high-quality incumbent solutions (?). Most closely related to this work is the GPU-accelerated state expansion framework for anytime exact DD search (?), leveraging GPU parallelism to accelerate the node expansion within a DD-based BnB solver.

6 Conclusion and Future Work

This paper presented the first parallel framework for exact Pareto frontier enumeration on decision diagrams for multi-objective discrete optimization. By targeting localized dominance filtering, the main computational bottleneck in DD-based enumeration, we developed parallel implementations for both multi-core CPUs and GPUs, along with hardware-aware scheduling and memory allocation strategies.

Our computational results show that parallelism can substantially improve enumeration performance. Multi-core CPU variants consistently outperform the serial baseline, while the GPU implementation delivers the best runtimes in most cases where it completes successfully. Overall, these findings show that DD-based Pareto frontier enumeration exposes significant parallelism and that GPUs are particularly well suited to its dominance-filtering workload.

At the same time, the experiments reveal a clear trade-off between speed and robustness. The GPU implementation inherently trades memory for parallelism: all candidate labels for an entire layer are materialized in GPU global memory simultaneously, so that threads can operate on disjoint segments without interfering with one another. When a layer's candidate set exceeds available GPU memory, the layer is partitioned into node sub-batches that are processed sequentially, which partially alleviates the issue but at the cost of reduced parallelism. A more advanced approach would avoid explicit materialization by generating candidate labels on-the-fly during the dominance check itself. However, this introduces a redundancy problem: multiple threads

would independently re-generate the same label whenever it is needed as a reference, creating a fundamental trade-off between memory consumption and compute redundancy that warrants further investigation. CPU-based methods are generally more robust due to their larger main-memory capacity, though their speedups are more modest and eventually saturate.

Several directions remain for future work. An immediate priority is investigating strategies that reduce GPU memory consumption without sacrificing parallelism, including the on-the-fly generation approach discussed above and more compact label representations. Another important direction is extending the GPU-accelerated coupled enumeration to support dynamic load-balancing across multi-GPU environments and hybrid CPU-GPU memory hierarchies. Further gains may also come from reducing host-device data transfer and exploring hybrid CPU-GPU schemes that balance throughput and memory capacity.

Overall, our results show that parallel DD-based enumeration is a promising direction for scaling exact multiobjective optimization and broadening the practical applicability of DD-based solvers.

Table 2: MOKP Pareto Enumeration Runtime Comparison Across TD/Coup CPU and GPU Methods (continued)

N	K	Method	Total	Enum.	(M, T)	N	K	Method	Total	Enum.	(M, T)
60	5	TD^+	1410	1409	(0, 1)	60	6	TD^+	2175	2174	(0, 4)
60	5	TD^+-4	932	932	(0, 1)	60	6	TD^+-4	1606	1605	(0, 4)
60	5	TD^+-8	860	859	(0, 1)	60	6	TD^+-8	1468	1467	(0, 4)
60	5	TD^+-16	834	833	(0, 1)	60	6	TD^+-16	1408	1408	(0, 4)
60	5	$Coup^+$	188	188	(0, 0)	60	6	$Coup^+$	884	883	(0, 2)
60	5	$Coup^+-4$	82	82	(0, 0)	60	6	$Coup^+-4$	597	596	(0, 0)
60	5	$Coup^+-8$	64	64	(0, 0)	60	6	$Coup^+-8$	450	450	(0, 0)
60	5	$Coup^+-16$	59	59	(0, 0)	60	6	$Coup^+-16$	362	362	(0, 0)
60	5	TD^+-GPU	33	31	(0, 0)	60	6	TD^+-GPU	206	204	(0, 0)
60	7	TD^+	2786	2785	(0, 8)	70	3	TD^+	16	15	(0, 0)
60	7	TD^+-4	1867	1866	(0, 8)	70	3	TD^+-4	12	11	(0, 0)
60	7	TD^+-8	1725	1725	(0, 8)	70	3	TD^+-8	10	10	(0, 0)
60	7	TD^+-16	1674	1673	(0, 8)	70	3	TD^+-16	10	9	(0, 0)
60	7	$Coup^+$	1307	1307	(0, 4)	70	3	$Coup^+$	5	4	(0, 0)
60	7	$Coup^+-4$	987	986	(0, 3)	70	3	$Coup^+-4$	3	2	(0, 0)
60	7	$Coup^+-8$	738	738	(0, 3)	70	3	$Coup^+-8$	3	2	(0, 0)
60	7	$Coup^+-16$	668	668	(0, 3)	70	3	$Coup^+-16$	3	2	(0, 0)
60	7	TD^+-GPU	333	331	(4, 0)	70	3	TD^+-GPU	4	1	(0, 0)
70	4	TD^+	734	733	(0, 0)	70	5	TD^+	2872	2871	(0, 7)
70	4	TD^+-4	516	515	(0, 0)	70	5	TD^+-4	2068	2067	(0, 6)
70	4	TD^+-8	482	481	(0, 0)	70	5	TD^+-8	1890	1889	(0, 6)
70	4	TD^+-16	464	463	(0, 0)	70	5	TD^+-16	1821	1821	(0, 6)
70	4	$Coup^+$	73	72	(0, 0)	70	5	$Coup^+$	779	778	(0, 2)
70	4	$Coup^+-4$	35	34	(0, 0)	70	5	$Coup^+-4$	368	367	(0, 2)
70	4	$Coup^+-8$	27	27	(0, 0)	70	5	$Coup^+-8$	513	512	(0, 1)
70	4	$Coup^+-16$	26	25	(0, 0)	70	5	$Coup^+-16$	677	677	(0, 0)
70	4	TD^+-GPU	11	8	(0, 0)	70	5	TD^+-GPU	104	102	(2, 0)
70	6	TD^+	—	—	(0, 10)	70	7	TD^+	—	—	(0, 10)
70	6	TD^+-4	—	—	(0, 10)	70	7	TD^+-4	—	—	(0, 10)
70	6	TD^+-8	—	—	(0, 10)	70	7	TD^+-8	3429	3429	(0, 9)
70	6	TD^+-16	—	—	(0, 10)	70	7	TD^+-16	3338	3337	(0, 9)
70	6	$Coup^+$	1500	1499	(0, 5)	70	7	$Coup^+$	1667	1666	(0, 7)
70	6	$Coup^+-4$	1320	1319	(0, 3)	70	7	$Coup^+-4$	1370	1370	(0, 6)
70	6	$Coup^+-8$	1034	1034	(0, 3)	70	7	$Coup^+-8$	1433	1432	(0, 5)
70	6	$Coup^+-16$	861	860	(0, 3)	70	7	$Coup^+-16$	1608	1608	(0, 4)
70	6	TD^+-GPU	343	341	(4, 0)	70	7	TD^+-GPU	382	379	(7, 0)

Table 3: MOSPP Pareto Enumeration Runtime Comparison Across TD/Coup CPU and GPU Methods

N	K	Method	Total	Enum.	(M, T)	N	K	Method	Total	Enum.	(M, T)
150	3	TD	11	4	(0, 0)	150	4	TD	22	13	(0, 0)
150	3	TD-4	9	3	(0, 0)	150	4	TD-4	12	5	(0, 0)
150	3	TD-8	7	2	(0, 0)	150	4	TD-8	10	4	(0, 0)
150	3	TD-16	8	2	(0, 0)	150	4	TD-16	9	3	(0, 0)
150	3	Coup	13	4	(0, 0)	150	4	Coup	12	7	(0, 0)
150	3	Coup-4	8	2	(0, 0)	150	4	Coup-4	10	4	(0, 0)
150	3	Coup-8	8	2	(0, 0)	150	4	Coup-8	9	3	(0, 0)
150	3	Coup-16	7	2	(0, 0)	150	4	Coup-16	7	3	(0, 0)
150	3	TD-GPU	8	1	(0, 0)	150	4	TD-GPU	8	2	(0, 0)
150	5	TD	61	50	(0, 0)	150	6	TD	128	123	(0, 0)
150	5	TD-4	26	14	(0, 0)	150	6	TD-4	42	36	(0, 0)
150	5	TD-8	19	11	(0, 0)	150	6	TD-8	30	24	(0, 0)
150	5	TD-16	13	6	(0, 0)	150	6	TD-16	23	20	(0, 0)
150	5	Coup	30	21	(0, 0)	150	6	Coup	47	43	(0, 0)
150	5	Coup-4	19	10	(0, 0)	150	6	Coup-4	22	18	(0, 0)
150	5	Coup-8	15	6	(0, 0)	150	6	Coup-8	18	15	(0, 0)
150	5	Coup-16	17	7	(0, 0)	150	6	Coup-16	22	15	(0, 0)
150	5	TD-GPU	7	2	(2, 0)	150	6	TD-GPU	7	2	(0, 0)
150	7	TD	619	614	(0, 0)	200	3	TD	174	73	(9, 0)
150	7	TD-4	173	168	(0, 0)	200	3	TD-4	162	36	(9, 0)
150	7	TD-8	118	111	(0, 0)	200	3	TD-8	75	14	(9, 0)
150	7	TD-16	89	86	(0, 0)	200	3	TD-16	68	12	(8, 1)
150	7	Coup	196	191	(0, 0)	200	3	Coup	104	38	(9, 0)
150	7	Coup-4	89	86	(0, 0)	200	3	Coup-4	88	27	(9, 0)
150	7	Coup-8	84	80	(0, 0)	200	3	Coup-8	78	16	(8, 1)
150	7	Coup-16	74	71	(0, 0)	200	3	Coup-16	103	33	(8, 1)
150	7	TD-GPU	13	8	(0, 0)	200	3	TD-GPU	–	–	(10, 0)
200	4	TD	307	271	(9, 0)						
200	4	TD-4	106	69	(9, 0)						
200	4	TD-8	89	40	(9, 0)						
200	4	TD-16	69	25	(9, 0)						
200	4	Coup	168	132	(9, 0)						
200	4	Coup-4	72	41	(9, 0)						
200	4	Coup-8	60	26	(9, 0)						
200	4	Coup-16	47	18	(9, 0)						
200	4	TD-GPU	–	–	(10, 0)						

Table 4: MOTSP Pareto Enumeration Runtime Comparison Across TD/Coup CPU and GPU Methods

N	K	Method	Total	Enum.	(M, T)	N	K	Method	Total	Enum.	(M, T)
15	4	TD	202	202	(0, 0)	15	5	TD	1926	1926	(0, 5)
15	4	TD-4	50	50	(0, 0)	15	5	TD-4	1099	1099	(0, 0)
15	4	TD-8	26	26	(0, 0)	15	5	TD-8	567	567	(0, 0)
15	4	TD-16	14	14	(0, 0)	15	5	TD-16	322	322	(0, 0)
15	4	Coup	23	23	(0, 0)	15	5	Coup	340	339	(0, 0)
15	4	Coup-4	7	7	(0, 0)	15	5	Coup-4	115	115	(0, 0)
15	4	Coup-8	5	5	(0, 0)	15	5	Coup-8	84	83	(0, 0)
15	4	Coup-16	4	3	(0, 0)	15	5	Coup-16	78	77	(0, 0)
15	4	TD-GPU	6	4	(0, 0)	15	5	TD-GPU	25	23	(6, 0)
15	6	TD	–	–	(0, 10)	15	7	TD	–	–	(0, 10)
15	6	TD-4	3415	3415	(0, 9)	15	7	TD-4	–	–	(0, 10)
15	6	TD-8	2664	2664	(0, 4)	15	7	TD-8	–	–	(0, 8)
15	6	TD-16	1873	1872	(0, 2)	15	7	TD-16	–	–	(0, 10)
15	6	Coup	1704	1704	(0, 1)	15	7	Coup	–	–	(0, 10)
15	6	Coup-4	883	883	(0, 0)	15	7	Coup-4	2551	2551	(0, 7)
15	6	Coup-8	785	784	(0, 0)	15	7	Coup-8	1426	1426	(0, 8)
15	6	Coup-16	785	785	(0, 0)	15	7	Coup-16	1263	1263	(0, 9)
15	6	TD-GPU	–	–	(10, 0)	15	7	TD-GPU	–	–	(10, 0)